บทที่ 3

การประสานเวลาของโพรเซส

ในระบบคอมพิวเตอร์แบบมัลติโปรแกรมมิ่ง ซึ่งอนุญาตให้มีโพรเซสหลายโพรเซสทำงานอยู่ในระบบพร้อมกันนั้น โพรเซสแต่ละตัวอาจทำงานอย่างเป็นอิสระจากกัน หรืออาจมีความเกี่ยวข้องและต้องพึ่งพากันในการดำเนินงานก็ได้ ทั้งนี้ไม่ว่าระบบจะประกอบด้วยหน่วยประมวลผลกลางเพียงตัวเดียวหรือหลายตัวก็ตาม ความสามารถในการจัดการกับโพรเซสที่ทำงานพร้อมกันถือเป็นสิ่งจำเป็น

สำหรับระบบที่มีเพียงหน่วยประมวลผลกลางเดียว โพรเซสต่าง ๆ จะผลัดกันใช้หน่วยประมวลผลกลางตามลำดับเวลาที่กำหนด ซึ่งเกิดขึ้นด้วยความถี่สูงหลายครั้งต่อวินาที ทำให้ผู้ใช้งานรู้สึกว่าโพรเซสเหล่านั้นทำงานพร้อมกัน ทั้งที่ในความเป็นจริงแล้วเป็นเพียงการสลับเวลาการทำงานระหว่างโพรเซสเท่านั้น

ในทางตรงกันข้าม หากระบบมีหน่วยประมวลผลกลางมากกว่าหนึ่งหน่วย โพรเซสต่าง ๆ สามารถทำงานได้จริงในเวลาเดียวกันบนหน่วยประมวลผลกลางแต่ละตัว ซึ่งลักษณะเช่นนี้เรียกว่า **ภาวะพร้อมกัน (concurrency)**

อย่างไรก็ตาม การที่โพรเซสหลายตัวทำงานในระบบพร้อมกัน อาจก่อให้เกิดปัญหาได้ หากโพรเซสเหล่านั้นมีความต้องการเข้าใช้ทรัพยากรเดียวกันในเวลาเดียวกัน เช่น การเข้าใช้ตำแหน่งหน่วยความจำเดียวกันพร้อมกัน อาจนำไปสู่ความผิดพลาดในการเขียนหรืออ่านข้อมูลจากหน่วยความจำที่ใช้ร่วมกัน ซึ่งอาจทำให้ผลลัพธ์ของการประมวลผลคลาดเคลื่อนหรือไม่ถูกต้องตามที่ควรจะเป็น

3.1 แนวคิดการประสานเวลาของโพรเซส

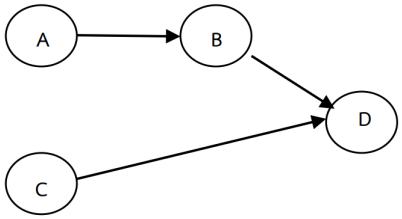
โดยทั่วไปแล้ว โพรเซสแต่ละตัวต้องการการทำงานที่เป็นอิสระจากกัน ราวกับว่ามีเพียงโพรเซสเดียวที่กำลังประมวลผลอยู่ในระบบ ลักษณะของการทำงานแบบนี้เรียกว่า **อะซิงโครนัส (Asynchronous)** อย่างไรก็ตาม ในความเป็นจริง ระบบคอมพิวเตอร์มักมีหลายโพรเซสทำงานพร้อมกันในเวลาเดียวกัน และโพรเซสเหล่านี้อาจต้องใช้ทรัพยากรที่มีอยู่อย่างจำกัดร่วมกัน เช่น ตำแหน่งในหน่วยความจำ หรืออุปกรณ์นำเข้าหรือส่งออก (I/O devices)

ด้วยเหตุนี้ ระบบปฏิบัติการจึงมีบทบาทสำคัญในการจัดการให้โพรเซสต่าง ๆ สามารถทำงานพร้อมกันได้อย่างมีประสิทธิภาพ ซึ่งลักษณะเช่นนี้เรียกว่า **การประมวลผลพร้อมกัน (concurrent processing)** นอกจากนี้ ยังต้องส่งเสริมให้โพรเซสสามารถ**ทำงานร่วมกัน (Cooperating)** ได้ โดยการใช้ทรัพยากรร่วมกันอย่างปลอดภัยและเป็นระเบียบ ซึ่งไม่เพียงช่วยให้การทำงานของระบบมีประสิทธิภาพสูงขึ้นเท่านั้น แต่ยังช่วยให้การพัฒนาโปรแกรมมีความสะดวกและยืดหยุ่นมากยิ่งขึ้น

ในการที่โพรเซสหลายตัวจะสามารถทำงานร่วมกันได้อย่างมีประสิทธิภาพนั้น จำเป็นต้องมี**การสื่อสารระหว่างโพรเซส (interprocess communication: IPC)** เพื่อแลกเปลี่ยนข้อมูลและสถานะการทำงานระหว่างกัน และต้องอาศัย**การประสานเวลา (process synchronization)** อย่างถูกต้อง โดยการประสาน

เวลาดังกล่าวหมายถึง การจัดการให้โพรเซสที่ต้องพึ่งพากันสามารถทำงานได้อย่างสอดคล้อง เช่น การที่โพรเซสหนึ่งต้องรอให้โพรเซสอื่นทำงานเสร็จก่อน หรือการเข้าถึงทรัพยากรที่ใช้ร่วมกันในลำดับที่ปลอดภัย

ให้นิสิตพิจารณารูปที่ 1.1 จะเห็นว่าโพรเซส B จะเริ่มทำงานได้ก็ต่อเมื่อโพรเซส A ทำงานเสร็จเรียบร้อยแล้ว (ซึ่งต่างกับโพรเซส C ที่ไม่ต้องรอโพรเซสใดเลยก็สามารถทำงานได้ทันที และไม่มีความเกี่ยวข้องกับโพรเซส A และ B เลย) ในทำนองเดียวกัน โพรเซส D จะทำงานได้ ก็ต่อเมื่อมีการทำงานโพรเซส B และโพรเซส C ทำงานเสร็จเรียบร้อยแล้วนั่นเอง



รูปที่ 3.1: การประสานเวลาของโพรเซส

3.2 ภาวะการเกิดพร้อมกัน

ในระบบคอมพิวเตอร์ ไม่ว่าจะเป็นระบบที่ทำงานบนหน่วยประมวลผลกลางเดียว หรือระบบที่มีหน่วยประมวลผลกลางหลายตัวก็ตาม มักมีความจำเป็นต้องใช้ทรัพยากรร่วมกันระหว่างโพรเซส ซึ่งทรัพยากรบางประเภทนั้นสามารถให้บริการแก่โพรเซสได้เพียงครั้งละหนึ่งเท่านั้น ไม่สามารถใช้งานพร้อมกันได้ โพรเซสที่ต้องการใช้ทรัพยากรเดียวกันจึงจำเป็นต้องผลัดกันใช้งานตามลำดับ ตัวอย่างของทรัพยากรลักษณะนี้ ได้แก่ เครื่องพิมพ์ หรืออุปกรณ์ส่งออกอื่น ๆ ทรัพยากรประเภทนี้เรียกว่า **ทรัพยากรที่ไม่สามารถใช้ร่วมกันได้ (non-dedicated resources)**

เมื่อมีโพรเซสหนึ่งกำลังใช้ทรัพยากรอยู่ โพรเซสอื่น ๆ จะไม่สามารถเข้าถึงทรัพยากรนั้นได้จนกว่าโพรเซสแรกจะสิ้นสุดการใช้งาน การจัดการการเข้าถึงทรัพยากรเช่นนี้มีความสำคัญเพื่อป้องกันไม่ให้เกิดความผิดพลาดในการทำงาน โดยเฉพาะอย่างยิ่งเมื่อโพรเซสต่าง ๆ ใช้ทรัพยากรหรือข้อมูลชุดเดียวกัน

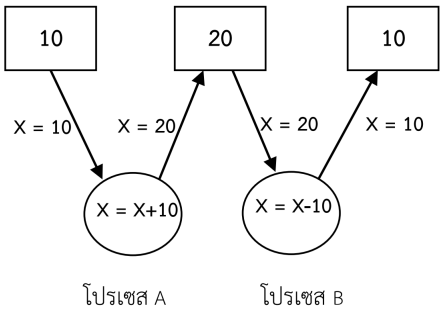
ปัญหาหนึ่งที่อาจเกิดขึ้นจากการใช้ทรัพยากรหรือข้อมูลร่วมกันระหว่างโพรเซสหลายตัว คือการที่โพรเซสทำงานพร้อมกันโดยไม่ได้มีการควบคุมอย่างเหมาะสม อาจทำให้ผลลัพธ์ของการประมวลผลผิดพลาดได้ ตัวอย่างเช่น สมมุติให้มีโพรเซสสองตัว คือ **โพรเซส A** และ **โพรเซส B** ทำงานในเวลาใกล้เคียงกัน และใช้ตัวแปรข้อมูลเดียวกัน คือ `X` ซึ่งมีค่าเริ่มต้นเท่ากับ 10 โดยที่:

- โพรเซส A ทำการเพิ่มค่า `X` ขึ้น 10 หน่วย โดยคำสั่ง `X = X + 10`
- โพรเซส B ทำการลดค่า `X` ลง 10 หน่วย โดยคำสั่ง `X = X - 10`

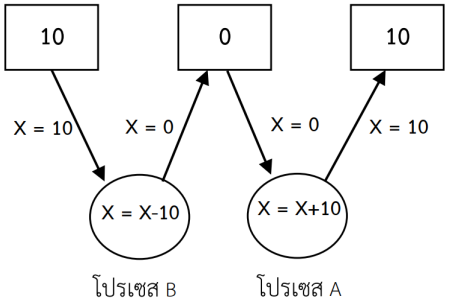
หากทั้งสองโพรเซสทำงานโดยไม่มีการจัดการเพื่อหลีกเลี่ยงการชนกัน หรือการประสานเวลา (synchronization) อย่างเหมาะสม อาจทำให้ค่าที่ได้สุดท้ายของ `X` ไม่เป็นไปตามที่ควรจะเป็น และทำให้เกิดความผิดพลาดในการทำงานของระบบโดยรวม

จากรูปที่ 1.2 โพรเซส A ทำงานก่อนโพรเซส B โดยอ่านค่า X เข้ามาซึ่งมีค่าเป็น 10 แล้วเพิ่มเข้าไปอีก 10 ทำให้ X มีค่าเป็น 20 แล้วเขียนลงหน่วยความจำ หลังจากนั้นโพรเซส B จึงอ่านค่า X เข้ามาซึ่งมีค่าเป็น 20 แล้วลบค่าออกไป 10 ทำให้ X มีค่าเป็น 10 ลงหน่วยความจำ

ในทางกลับกัน จากรูปที่ 1.3 การทำงานจะคล้ายกับรูปที่ 1.2 และได้ผลลัพธ์เหมือนกัน แต่โพรเซส B ทำงานก่อนโพรเซส A โดยอ่านค่า X เข้ามาซึ่งมีค่าเป็น 10 แล้วลบค่าออกไป 10 ทำให้ X มีค่าเป็น 0 จาก

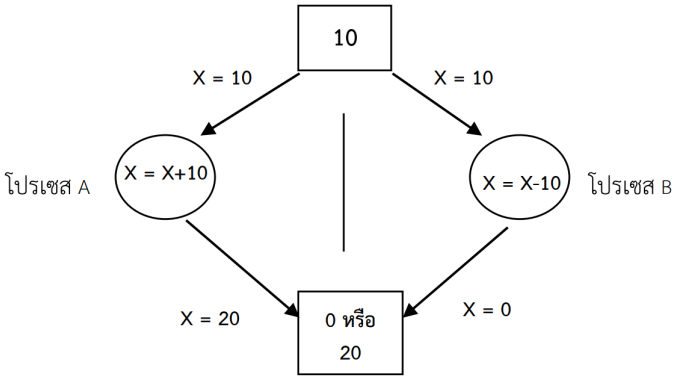


รูปที่ 3.2: โพรเซส A ทำงานเสร็จก่อนโพรเซส B



รูปที่ 3.3: โพรเซส B ทำงานเสร็จก่อนโพรเซส A

นั้นเขียนลงหน่วยความจำ หลังจากนั้นโพรเซส A จึงอ่านค่า X เข้ามาซึ่งมีค่าเป็น 0 แล้วเพิ่มค่าเข้าไป 10 ทำให้ X มีค่าเป็น 10 แล้วเขียนค่า X ลงหน่วยความจำ



รูปที่ 3.4: โพรเซส A และโพรเซส B ทำงานพร้อมกัน

จากตัวอย่างโพรเซส A และโพรเซส B ที่ต้องการใช้งานตัวแปรร่วมกันในหน่วยความจำ กล่าวคือ ตัวแปร **X** ซึ่งมีค่าเริ่มต้นเท่ากับ 10 โดยโพรเซส A ทำการเพิ่มค่า **X** ขึ้น 10 หน่วย (X = X + 10) และโพรเซส B ทำการลดค่า **X** ลง 10 หน่วย (X = X - 10) หากโพรเซสทั้งสองทำงานเรียงลำดับอย่างเหมาะสม เช่น โพรเซส A ทำงานจนเสร็จแล้วจึงถึงคิวโพรเซส B หรือในทางกลับกัน จะทำให้ค่าของ **X** ที่ได้เป็นผลลัพธ์ที่ถูกต้องและสามารถคาดการณ์ได้

อย่างไรก็ตาม ในสถานการณ์ที่ทั้งสองโพรเซสทำงาน**พร้อมกัน** ดังเช่นในรูปที่ 1.4 จะเกิดปัญหาขึ้นได้ เพราะแม้ว่าทั้งสองโพรเซสจะเห็นค่าเริ่มต้นของ **X** เป็น 10 เท่ากัน แต่โพรเซส A จะคำนวณได้ค่าใหม่เป็น 20 (10 + 10) ส่วนโพรเซส B จะคำนวณได้ค่าใหม่เป็น 0 (10 - 10) ดังนั้น ผลลัพธ์สุดท้ายที่ถูกเขียนลง

หน่วยความจำจะขึ้นอยู่กับว่าโพรเซสใดทำงานเสร็จก่อน

หากโพรเซส A ทำงานเสร็จก่อน ก็จะเขียนค่า `X = 20` ลงหน่วยความจำ แต่ถ้าโพรเซส B ทำงานเสร็จก่อน ก็จะเขียนค่า `X = 0` แทน ซึ่งหมายความว่าผลลัพธ์สุดท้ายของ `X` จะไม่แน่นอน และอาจแตกต่างกันในแต่ละครั้งที่รันโปรแกรม โดยขึ้นอยู่กับลำดับเวลาของการทำงานของแต่ละโพรเซส ลักษณะเช่นนี้เรียกว่า **สภาวะเงื่อนไขการแย่งชิง (race condition)**

สภาวะเงื่อนไขการแย่งชิงจะเกิดขึ้นเมื่อโพรเซสตั้งแต่สองตัวขึ้นไปทำงานกับทรัพยากรที่ใช้ร่วมกัน เช่น หน่วยความจำ สื่อจัดเก็บข้อมูล หรือไฟล์ต่าง ๆ และสามารถอ่านหรือเขียนข้อมูลได้ในเวลาเดียวกัน โดยไม่มีการควบคุมลำดับอย่างเหมาะสม ซึ่งนำไปสู่ผลลัพธ์ที่ผิดพลาดหรือไม่สามารถคาดการณ์ได้

เพื่อหลีกเลี่ยงปัญหานี้ ระบบจำเป็นต้องรับประกันว่า **ในเวลาใดเวลาหนึ่ง จะมีเพียงโพรเซสเดียวเท่านั้นที่สามารถจัดการกับทรัพยากรหรือข้อมูลตัวแปรนั้นได้** โดยต้องอาศัยกลไกบางอย่างในการควบคุมการเข้าถึง เช่น การประสานเวลา (process synchronization) เพื่อให้โพรเซสทำงานได้อย่างเป็นลำดับและปลอดภัย

3.3 การแก้ปัญหาส่วนวิกฤติ

ปัญหาที่เกิดจากสถานการณ์ของ**เงื่อนไขการแย่งชิง (race condition)** มักมีสาเหตุมาจากการที่โพรเซสหลายตัวเข้าใช้ทรัพยากรร่วมกัน โดยไม่มีการประสานการทำงานอย่างเหมาะสม แต่ละโพรเซสต่างทำงานตามคำสั่งของตนเองไปเรื่อย ๆ จนกระทั่งถึงจุดที่มีการใช้งานทรัพยากรซึ่งต้องใช้ร่วมกันกับโพรเซสอื่น ส่งผลให้ผลลัพธ์ที่ได้เกิดความผิดพลาด และไม่สามารถคาดการณ์ได้

เพื่อลดปัญหาดังกล่าว ระบบควรมีการ **ประสานการเข้าทำงานในบริเวณที่โพรเซสใช้งานทรัพยากรร่วมกัน** ได้อย่างถูกต้องและปลอดภัย แนวคิดที่ใช้ในการอธิบายบริเวณของโปรแกรมซึ่งโพรเซสเข้าไปเปลี่ยนแปลงข้อมูล แก้ไขตาราง บันทึกข้อมูล หรือทำงานอื่น ๆ ที่ต้องใช้ทรัพยากรร่วมกันนั้น เรียกว่า **ส่วนวิกฤติ (critical section หรือ critical region)**

ให้พิจารณาระบบที่ประกอบด้วยโพรเซสจำนวน n ตัว คือ P_0, P_1, ..., P_{n-1} ซึ่งแต่ละโพรเซสจะมีบางส่วนของโปรแกรมที่ต้องเข้าไปจัดการทรัพยากรหรือข้อมูลที่ใช้ร่วมกัน เรากำหนดคุณสมบัติที่สำคัญของระบบว่า **ขณะที่โพรเซสหนึ่งกำลังอยู่ในส่วนวิกฤติ จะต้องไม่มีโพรเซสอื่นใดสามารถเข้าไปอยู่ในส่วนวิกฤติพร้อมกันได้** กล่าวคือ ต้องมีการป้องกันการเข้าถึงพร้อมกันของหลายโพรเซสในช่วงเวลานั้น

การป้องกันไม่ให้โพรเซสหลายตัวเข้าใช้งานส่วนวิกฤติในเวลาเดียวกัน เรียกว่า **การกีดกัน (mutual exclusion)** ซึ่งเป็นแนวทางหลักในการออกแบบกลไกเพื่อควบคุมการเข้าถึงทรัพยากรร่วมกัน โดยจะต้องมีกลไกหรือ **โปรโตคอล (protocol)** ที่กำหนดลำดับการทำงานอย่างเป็นระเบียบ ซึ่งโพรเซสต้องปฏิบัติตามเพื่อให้สามารถทำงานร่วมกันได้อย่างปลอดภัยและมีประสิทธิภาพ

การดำเนินงานของแต่ละโพรเซสเมื่อจะเข้าใช้งานส่วนวิกฤติจะต้องเป็นไปตามขั้นตอนต่อไปนี้:

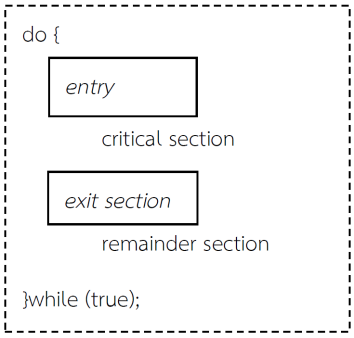
1. **ร้องขอเพื่อเข้าสู่ส่วนวิกฤติ** โดยตรวจสอบเงื่อนไขตามโปรโตคอลที่กำหนด
2. **เข้าทำงานในส่วนวิกฤติ** เมื่อได้รับอนุญาต
3. **ออกจากส่วนวิกฤติ** เมื่อเสร็จสิ้นการใช้งานทรัพยากร
4. **ทำงานในส่วนอื่นของโปรแกรม** ซึ่งไม่เกี่ยวข้องกับโพรเซสอื่น

ทั้งนี้ แนวทางการแก้ปัญหาในส่วนวิกฤติจะต้องเป็นไปตาม **เงื่อนไขสำคัญสามประการ** ดังนี้:

1. **การกีดกัน (Mutual Exclusion)**: หากโพรเซส P_i กำลังทำงานอยู่ในส่วนวิกฤติ จะต้องไม่มีโพรเซสอื่นสามารถเข้าใช้งานในส่วนวิกฤติได้ในเวลาเดียวกัน กล่าวคือ ณ เวลาใดเวลาหนึ่ง มีเพียงโพรเซสเดียวเท่านั้นที่อยู่ในส่วนวิกฤติได้ (ดังรูปที่ 1.5)

2. **ความก้าวหน้า (Progress)**: หากไม่มีโพรเซสใดอยู่ในส่วนวิกฤติ แต่มีโพรเซสหนึ่งหรือมากกว่าต้องการเข้าสู่ส่วนวิกฤติ การเลือกว่าจะให้โพรเซสใดเข้าสู่ส่วนวิกฤติต่อไป ต้องขึ้นอยู่กับการตัดสินใจของโพรเซสที่อยู่ในสถานะพร้อม ไม่ใช่การรอแบบไม่มีกำหนด

3. **ขอบเขตเวลาการรอ (Bounded Waiting)**: โพรเซสที่ร้องขอเพื่อเข้าสู่ส่วนวิกฤติจะต้องไม่รอเป็นระยะเวลาไม่จำกัด กล่าวคือ ต้องมีขอบเขตเวลารอที่ชัดเจนตั้งแต่เวลาที่โพรเซสร้องขอ จนถึงเวลาที่สามารถเข้าใช้งานส่วนวิกฤติได้จริง



รูปที่ 3.5: การดำเนินการของโพรเซสในการเข้าใช้ในส่วนวิกฤติ

ทั้งนี้ ภายใต้ข้อสมมติว่า แต่ละโพรเซสกำลังทำงานด้วยความเร็วที่ไม่เป็นศูนย์ แต่**ไม่กำหนดสมมติฐานเกี่ยวกับความเร็วสัมพัทธ์** ของโพรเซสทั้งหมด n ตัว แนวคิดในการออกแบบขั้นตอนวิธีเพื่อแก้ปัญหาส่วนวิกฤติ จึงต้องคำนึงถึงเงื่อนไขสำคัญทั้งสามประการ ได้แก่ การกีดกัน (Mutual Exclusion), ความก้าวหน้า (Progress), และขอบเขตเวลาการรอ (Bounded Waiting) เพื่อให้โพรเซสสามารถทำงานร่วมกันได้อย่างถูกต้อง

จากเงื่อนไขดังกล่าว ได้มีการพัฒนาโครงสร้างการทำงานในแต่ละโพรเซส โดยเพิ่มส่วนเริ่มต้นก่อนเข้าสู่**ส่วนวิกฤติ (critical section)** เพื่อควบคุมการเข้าใช้ทรัพยากรให้เป็นไปตามหลักการกีดกัน และส่วนปิดท้ายหลังจากออกจากส่วนวิกฤติ เพื่อปล่อยให้โพรเซสอื่นสามารถเข้าสู่ส่วนวิกฤติได้ต่อไป ซึ่งเป็นการรองรับเงื่อนไขของความก้าวหน้าและการจำกัดเวลารอ

ในส่วนเริ่มต้นของโพรเซส จะมีการตรวจสอบว่า มีโพรเซสอื่นใดกำลังทำงานอยู่ในส่วนวิกฤติหรือไม่ หากไม่มี โพรเซสจะสามารถเข้าสู่ส่วนวิกฤติได้ทันที แต่หากมีโพรเซสอื่นอยู่ โพรเซสจะต้องรอจนกว่าจะได้รับสิทธิ์ในการเข้าใช้งาน ซึ่งนำไปสู่คำถามสำคัญว่า**โพรเซสที่ไม่สามารถเข้าสู่ส่วนวิกฤติได้ในขณะนั้น ควรจะรออย่างไร** เพื่อไม่ให้กระทบต่อประสิทธิภาพและความถูกต้องของระบบ

แนวคิดในการจัดการการรอเพื่อเข้าสู่ส่วนวิกฤติ สามารถแบ่งออกได้เป็นสองแนวทางหลัก ได้แก่:

- **เคอร์เนลแบบมีสิทธิได้ก่อน (Preemptive kernels)** ในแนวทางนี้ โพรเซสจะทำการตรวจสอบเงื่อนไขก่อนเข้าสู่ส่วนวิกฤติ หากยังไม่สามารถผ่านเงื่อนไขได้ โพรเซสนั้นจะต้องวนลูปทำการตรวจสอบซ้ำอยู่ตลอดเวลา จนกว่าจะมีสิทธิ์ในการเข้าสู่ส่วนวิกฤติ เมื่อโพรเซสใดได้รับสิทธิ์ ก็สามารถใช้

การขัดจังหวะ (Interrupt) เพื่อเข้าสู่ส่วนวิกฤติได้ทันที แนวทางนี้ทำให้โพรเซสที่รอจะใช้ทรัพยากรต้องทำงานซ้ำโดยไม่หยุดพัก ซึ่งอาจสิ้นเปลืองทรัพยากรของระบบโดยไม่จำเป็น

- **เคอร์เนลแบบไม่ให้สิทธิก่อน (Non-preemptive kernels)** แนวทางนี้จะใช้กลไกการ**บล็อก (Block)** โพรเซสที่ยังไม่สามารถเข้าสู่ส่วนวิกฤติได้ โดยให้โพรเซสเหล่านี้ไปอยู่ใน **แถวรอ (Waiting Queue)** ตามเงื่อนไขที่กำหนดไว้ โดยไม่ต้องแย่งชิงทรัพยากรอย่างต่อเนื่อง เมื่อไม่มีโพรเซสใดอยู่ในส่วนวิกฤติแล้ว ระบบจะ **ปลุก (Wake up)** โพรเซสจากแถวรอ เพื่อให้เข้าสู่ส่วนวิกฤติต่อไป แนวทางนี้เหมาะสมกับระบบที่ต้องการความเสถียร เช่น ระบบเรียลไทม์ และช่วยลดความเสี่ยงจากการแย่งชิงที่ไม่จำเป็นได้ดี

3.4 การควบคุมการทำงานโดยวิธีการทางซอฟต์แวร์

การควบคุมการทำงานของโพรเซสในระบบคอมพิวเตอร์ที่มีการประมวลผลแบบพร้อมกัน สามารถทำได้โดย **วิธีการทางซอฟต์แวร์** ซึ่งเป็นแนวทางที่ใช้การออกแบบขั้นตอนวิธี (algorithm) เพื่อให้โพรเซสสามารถเข้าสู่ส่วนวิกฤติ (critical section) ได้อย่างปลอดภัย โดยต้องมีคุณสมบัติสำคัญคือ **ไม่เกิดร่วมกัน (mutual exclusion)** นั่นคือ ต้องไม่มีโพรเซสมากกว่าหนึ่งตัวที่สามารถอยู่ในส่วนวิกฤติได้ในเวลาเดียวกัน

ในบรรดาอัลกอริทึมที่ใช้แก้ปัญหาส่วนวิกฤติด้วยวิธีซอฟต์แวร์ มีหลายแนวทางที่น่าสนใจ ซึ่งหนึ่งในอัลกอริทึมเบื้องต้นที่มีความสำคัญทางประวัติศาสตร์ คือ

3.4.1 อัลกอริทึมของเดกเกอร์ (Dekker's Algorithm)

อัลกอริทึมของเดกเกอร์ ถือเป็นหนึ่งในอัลกอริทึมแรกเริ่มที่ถูกพัฒนาขึ้นเพื่อแก้ปัญหาการเข้าถึงส่วนวิกฤติโดยไม่ต้องพึ่งพาฮาร์ดแวร์ในการควบคุมการเข้าถึงทรัพยากร อัลกอริทึมนี้ออกแบบมาเพื่อให้ทำงานได้กับระบบที่มีโพรเซส 2 ตัว ได้แก่ `P0` และ `P1` ซึ่งสามารถทำงานพร้อมกันได้

วัตถุประสงค์หลักของอัลกอริทึมนี้คือ เพื่อให้โพรเซสทั้งสองสามารถเข้าสู่ส่วนวิกฤติได้โดยไม่ละเมิดคุณสมบัติของการไม่เกิดร่วม และยังสนับสนุนให้เกิดความก้าวหน้า (progress) และขอบเขตการรอ (bounded waiting) ตามหลักของการออกแบบโพรโตคอลควบคุมส่วนวิกฤติ

จากตัวอย่างตามรูปที่ 1.6 แสดงให้เห็นการทำงานของโพรเซสสองตัวคือ `P0` และ `P1` โดยใช้อัลกอริทึมควบคุมการเข้าสู่ **ส่วนวิกฤติ (critical section)** ด้วยตัวแปร `turn` ซึ่งถูกกำหนดค่าเริ่มต้นให้เป็น 0 ซึ่งหมายความว่า โพรเซส `P0` จะสามารถเข้าสู่ส่วนวิกฤติได้ก่อน ขณะที่โพรเซส `P1` ต้องวนลูปรอ (busy wait) จนกว่า `P0` จะออกจากส่วนวิกฤติและเปลี่ยนค่าของ `turn` ให้เป็น 1

เมื่อ `P0` เสร็จสิ้นการทำงานในส่วนวิกฤติ และเปลี่ยนค่าตัวแปร `turn` เป็น 1 แล้ว โพรเซส `P1` จึงสามารถออกจากลูปรอ และเข้าสู่ส่วนวิกฤติได้เช่นกัน จากนั้นวงจรการผลัดกันเข้าสู่ส่วนวิกฤติระหว่าง `P0` และ `P1` ก็จะดำเนินต่อไป ผลลัพธ์ที่ได้คือ โพรเซสทั้งสองสามารถทำงานผลัดกันได้อย่างปลอดภัย โดยไม่มีการเข้าสู่ส่วนวิกฤติพร้อมกัน ซึ่งแสดงให้เห็นถึงการคงคุณสมบัติ **ไม่เกิดร่วมกัน (mutual exclusion)** ได้อย่างถูกต้อง

อย่างไรก็ตาม อัลกอริทึมนี้ยังมีข้อจำกัดที่สำคัญ โดยเฉพาะในกรณีที่เกิดสถานการณ์ **busy waiting** ตัวอย่างเช่น หากโพรเซส `P0` กำลังทำงานในส่วนวิกฤติ แล้วหมดเวลาการประมวลผล (timeout) ก่อนที่จะเปลี่ยนค่าของตัวแปร `turn` เป็น 1 โพรเซส `P1` ที่กำลังรอจะไม่สามารถเข้าสู่ส่วนวิกฤติได้ แม้ว่าในขณะนั้น `P0` จะไม่ได้ทำงานแล้วก็ตาม ส่งผลให้ `P1` ต้องวนลูปรอโดยไม่มีความคืบหน้า ทำให้ทรัพยากรถูกใช้โดยเปล่าประโยชน์

```
//flag[] is boolean array; and turn is an integer
    flag[0] = false
    flag[1] = false
    turn    = 0   // or 1
```

```
P0:
   flag[0] = true;
   while (flag[1] == true) {
      if (turn ≠ 0) {
         flag[0] = false;
         while (turn ≠ 0) {
           // busy wait
         }
         flag[0] = true;
      }
   }
    // critical section
   ...
   turn    = 1;
   flag[0] = false;
   // remainder section
```

```
P1:
   flag[1] = true;
   while (flag[0] == true) {
      if (turn ≠ 1) {
         flag[1] = false;
         while (turn ≠ 1) {
           // busy wait
         }
         flag[1] = true;
      }
   }
    // critical section
   ...
   turn    = 0;
   flag[1] = false;
   // remainder section
```

รูปที่ 3.6: การใช้อัลกอริทึมของเดกเกอร์ในการทำงานของโพรเซส P0 และ P1

อีกหนึ่งปัญหาที่พบได้จากอัลกอริทึมนี้คือ **Lock Step Synchronization** ซึ่งเป็นภาวะที่การเข้าทำงานในส่วนวิกฤติของโพรเซสจะต้องเป็นไปตามลำดับที่ตายตัว เช่น P0, P1, P0, P1, ... โดยไม่เปิดโอกาสให้โพรเซสใดข้ามรอบหรือเว้นการเข้าใช้งานได้ แม้ว่าในบางกรณี โพรเซสหนึ่งอาจไม่มีความจำเป็นต้องเข้าสู่ส่วนวิกฤติเลยก็ตาม ลักษณะนี้ทำให้ระบบขาดความยืดหยุ่นและประสิทธิภาพลดลง

3.4.2 อัลกอริทึมปีเตอร์สันโซลูชั่น

Peterson's Solution เป็นหนึ่งในอัลกอริทึมทางซอฟต์แวร์ที่ได้รับความนิยมและถือเป็นพื้นฐานในการแก้ไขปัญหาส่วนวิกฤติ (critical section problem) สำหรับโพรเซสในระบบคอมพิวเตอร์ อัลกอริทึมนี้ถูกออกแบบมาเพื่อควบคุมการทำงานของ **โพรเซสสองตัว** ที่ต้องผลัดกันเข้าใช้งานส่วนวิกฤติและกลับไปยังส่วนอื่นของโปรแกรม โดยยังคงคุณสมบัติการ **ไม่เกิดร่วม (mutual exclusion)** และสนับสนุนความก้าวหน้า (progress) รวมถึงขอบเขตเวลารอ (bounded waiting)

ในรูปที่ 1.7 แสดงตัวอย่างโพรเซสสองตัว ได้แก่ P0 และ P1 ซึ่งใช้ข้อมูลร่วมกัน โดยโพรเซสทั้งสองจะมีการใช้ตัวแปรร่วมกันสองค่าคือ:

- `flag[0]` และ `flag[1]`: ใช้ตรวจสอบว่าโพรเซสใดกำลังแสดงเจตนาจะเข้าสู่ส่วนวิกฤติ
- `turn`: ใช้ระบุว่าโพรเซสใดควรได้รับสิทธิ์เข้าสู่ส่วนวิกฤติก่อน

```
//flag[] is boolean array; and turn is an integer
flag[0]   = false;
flag[1]   = false;
turn;
```

```
P0: flag[0] = true;
    turn = 1;
    while (flag[1] == true && turn == 1)
    {
        // busy wait
    }
    // critical section
    ...
    // end of critical section
    flag[0] = false;
```

```
P1: flag[1] = true;
    turn = 0;
    while (flag[0] == true && turn == 0)
    {
        // busy wait
    }
    // critical section
    ...
    // end of critical section
    flag[1] = false;
```

รูปที่ 3.7: โครงสร้างภาษาที่ใช้อัลกอริทึมของปีเตอร์สันโซลูชั่นของโพรเซส P0 และ P1

เมื่อเริ่มการทำงาน ตัวแปร `flag[0]` และ `flag[1]` ถูกกำหนดค่าเริ่มต้นให้เป็น `false` แสดงว่าไม่มีโพรเซสใดต้องการเข้าสู่ส่วนวิกฤติ

หากโพรเซสหนึ่งต้องการเข้าสู่ส่วนวิกฤติ เช่น โพรเซส P0 จะทำการตั้งค่า `flag[0]` ให้เป็น `true` เพื่อแสดงความตั้งใจ จากนั้นตั้งค่า `turn` ให้เป็น 1 เพื่อให้สิทธิ์แก่โพรเซส P1 และจะเข้าสู่ส่วนวิกฤติได้ก็ต่อเมื่อ P1 ไม่แสดงเจตนาเข้าสู่ส่วนวิกฤติ (คือ `flag[1]` เป็น `false`) หรือ `turn` ไม่ใช่ 1

ในทางกลับกัน ถ้า P1 ต้องการเข้าสู่ส่วนวิกฤติ ก็จะตั้งค่า `flag[1]` เป็น `true` และตั้งค่า `turn` เป็น 0 แล้วจึงรอจนกว่าจะถึงเวลาที่เหมาะสมตามเงื่อนไขของอัลกอริทึม

หลังจากที่โพรเซสหนึ่งทำงานในส่วนวิกฤติเสร็จแล้ว จะทำการตั้งค่า `flag` ของตนเองกลับเป็น `false` เพื่อเปิดโอกาสให้โพรเซสอีกตัวหนึ่งเข้าสู่ส่วนวิกฤติได้ต่อไป

Peterson's Algorithm นี้สามารถรับประกันการไม่เกิดร่วมสำหรับโพรเซสสองตัว และยังสามารถขยายแนวคิดไปสู่การประยุกต์ใช้กับโพรเซสมากกว่าสองตัว (n โพรเซส) ได้อีกด้วย โดยใช้แนวคิดแบบลำดับชั้นในการควบคุมการเข้าถึงส่วนวิกฤติ

3.5 การควบคุมการทำงานโดยวิธีการทางฮาร์ดแวร์

การควบคุมการประมวลผลของโพรเซสที่ทำงานพร้อมกันและยังคงคุณสมบัติ**การไม่เกิดร่วม (mutual exclusion)** สามารถทำได้ด้วยวิธีการทางฮาร์ดแวร์ ซึ่งเป็นกลไกที่ฝังอยู่ในระดับของสถาปัตยกรรมของระบบคอมพิวเตอร์ โดยวิธีการที่นิยมใช้มีดังต่อไปนี้:

1. การปิดกั้น (Lock)
2. การปิดทางขัดจังหวะ (Disabling Interrupt)
3. คำสั่งทดสอบและเซ็ต (Test and Set Instruction)

3.5.1 การปิดกั้น (Lock)

แนวคิดของการปิดกั้นหรือการใช้**ล็อค (Lock)** คือ การควบคุมการเข้าใช้งาน**ส่วนวิกฤติ (critical section)** ของโพรเซส โดยอาศัยตัวแปรสถานะหรือตัวแปรล็อคเพื่อกำหนดว่า ขณะใดขณะหนึ่งมีโพรเซสใดอยู่ในส่วนวิกฤติหรือไม่ (ดังแสดงในรูปที่ 1.8)

เมื่อโพรเซสหนึ่งต้องการเข้าสู่ส่วนวิกฤติ จะต้องทำการตรวจสอบก่อนว่าส่วนวิกฤติถูกปิดกั้นอยู่หรือไม่ หากพบว่าส่วนวิกฤติยังไม่ถูกล็อค แสดงว่าไม่มีโพรเซสอื่นใช้งานอยู่ โพรเซสนั้นจึงสามารถเข้าไปทำงานได้ และก่อนเข้าสู่ส่วนวิกฤติจะต้องตั้งค่าล็อคเพื่อป้องกันไม่ให้โพรเซสอื่นเข้ามาใช้งานพร้อมกันได้ หลังจากที่โพรเซสทำงานในส่วนวิกฤติเสร็จเรียบร้อยแล้ว จะต้องปลดล็อค เพื่อเปิดโอกาสให้โพรเซสอื่นสามารถเข้าสู่ส่วนวิกฤติได้ต่อไป วิธีการทำงานเช่นนี้สามารถช่วยรับประกันคุณสมบัติของการ**ไม่เกิดร่วม (mutual exclusion)** ได้ในระดับหนึ่ง

แม้ว่าการใช้ล็อคจะดูเหมือนเป็นกลไกที่สามารถควบคุมได้โดยซอฟต์แวร์ (เช่น การใช้ตัวแปร `flag`) แต่ในความเป็นจริง การรับประกันว่าช่วงเวลาใดเวลาหนึ่งจะมีเพียงโพรเซสเดียวเท่านั้นที่สามารถเข้าใช้งานส่วนวิกฤติได้อย่างแท้จริง โดยไม่เกิดปัญหา **race condition** นั้น จำเป็นต้องอาศัยความสามารถของ**คำสั่งระดับฮาร์ดแวร์**

คำสั่งระดับฮาร์ดแวร์ที่ใช้ในบริบทนี้มักเป็นคำสั่งที่มีคุณสมบัติแบบ**อะตอมมิก (atomic operation)** คือสามารถดำเนินการทั้งอ่านและเขียนค่าของตัวแปรได้ภายในคำสั่งเดียว โดยไม่เปิดโอกาสให้โพรเซสอื่นแทรกกลางเข้ามาได้ระหว่างนั้น ซึ่งเป็นคุณสมบัติที่ทำได้เฉพาะในระดับของสถาปัตยกรรมฮาร์ดแวร์เท่านั้น

```
do {
    acquire lock
        critical section
    release lock
        remainder section
} while (TRUE);
```

รูปที่ 3.8: รูปแบบของคำสั่งปิดกั้น (Lock)

แม้ว่าแนวทางนี้จะช่วยให้โพรเซสสามารถทำงานพร้อมกันได้โดยไม่เกิดการแทรกแซงซึ่งกันและกันในส่วนวิกฤติ แต่ก็มีข้อจำกัดในกรณีที่มีโพรเซสมากกว่าหนึ่งตัวต้องการเข้าใช้งานส่วนวิกฤติพร้อมกัน และต่างก็ตรวจสอบสถานะของล็อคในเวลาใกล้เคียงกัน หากพบว่าล็อควาง ทั้งหมดก็อาจเข้าสู่ส่วนวิกฤติพร้อมกันได้ ซึ่งจะเป็นการละเมิดคุณสมบัติของการไม่เกิดร่วม และทำให้เกิดข้อผิดพลาดในการประมวลผล

3.5.2 การปิดการขัดจังหวะ

หนึ่งในสาเหตุสำคัญที่ทำให้เกิดปัญหาในการทำงานพร้อมกันของโพรเซสหลายตัว คือ การที่โพรเซสหนึ่งกำลังทำงานอยู่ในส่วนวิกฤติ (critical section) แล้วมีโพรเซสอื่นทำการ**ขอขัดจังหวะ (interrupt)** เพื่อเข้าไปทำงานในส่วนวิกฤติแทน ส่งผลให้โพรเซสเดิมต้องหยุดการทำงานกลางคัน ทั้งที่ยังดำเนินการไม่เสร็จสมบูรณ์ โพรเซสใหม่ที่เข้ามาแทนมีโอกาสเปลี่ยนแปลงข้อมูล หรือดำเนินการใด ๆ ที่ส่งผลกระทบต่อผลลัพธ์ของโพรเซสก่อนหน้าได้ ซึ่งอาจก่อให้เกิดปัญหาความผิดพลาดในการทำงานของระบบ

ในระบบที่มี**หน่วยประมวลผลกลางเดียว (Single Processor)** วิธีการหนึ่งที่สามารถใช้ควบคุมการเข้าถึงส่วนวิกฤติได้ คือ **การปิดทางขัดจังหวะ (disabling interrupt)** ซึ่งช่วยรับประกันได้ว่าชุดคำสั่งที่

อยู่ภายในส่วนวิกฤติจะไม่ถูกแทรกกลางโดยโพรเซสอื่น ระบบจะไม่เปลี่ยนบริบทการทำงานจนกว่าชุดคำสั่งดังกล่าวจะเสร็จสมบูรณ์

อย่างไรก็ตาม แนวทางนี้**ไม่สามารถใช้ได้อย่างมีประสิทธิภาพในระบบที่มีหลายหน่วยประมวลผล (multiprocessor systems)** เนื่องจากการสั่งปิดการขัดจังหวะในทุกหน่วยประมวลผลกลางใช้เวลาในการส่งสัญญาณนาน และอาจก่อให้เกิด **ความล่าช้า (latency)** ในระบบ ส่งผลให้ประสิทธิภาพโดยรวมลดลง นอกจากนี้ โพรเซสยังอาจถูกขัดจังหวะจากโพรเซสเซอร์ตัวอื่นที่ยังไม่ได้รับคำสั่งปิดการขัดจังหวะ จึงไม่สามารถรับประกันการไม่เกิดร่วมได้ในระบบมัลติโพรเซสเซอร์

อีกทั้ง วิธีนี้ยังมีข้อควรระวังที่สำคัญคือ **ไม่ควรอนุญาตให้โพรเซสใดสามารถปิดทางขัดจังหวะได้ด้วยตนเอง** เพราะหากโพรเซสนั้นไม่ปล่อยการขัดจังหวะกลับคืนให้ระบบ หรือเกิดข้อผิดพลาดระหว่างการประมวลผล อาจทำให้เกิดผลกระทบต่อระบบโดยรวม เช่น การละเมิดเวลาตอบสนอง การหยุดการทำงานของบริการสำคัญ หรือการบล็อกโพรเซสอื่นไม่ให้ทำงานต่อ

การปิดทางขัดจังหวะสามารถใช้ควบคุมการเข้าถึงส่วนวิกฤติได้อย่างปลอดภัยเฉพาะในระบบที่มีหน่วยประมวลผลกลางเพียงตัวเดียวเท่านั้น ในขณะที่ระบบที่มีหลายหน่วยประมวลผลกลางจำเป็นต้องใช้กลไกอื่นที่สามารถประสานการทำงานข้ามหน่วยประมวลผลได้อย่างมีประสิทธิภาพมากกว่า

3.5.3 คำสั่งทดสอบและเซต

คำสั่งทดสอบและเซต (Test-and-Set) เป็นคำสั่งระดับฮาร์ดแวร์ที่ถูกพัฒนาขึ้นเพื่อแก้ไขข้อจำกัดของวิธีการควบคุมส่วนวิกฤติแบบเดิม เช่น การใช้ **ล็อค (lock)** หรือการ **ปิดทางขัดจังหวะ (disable interrupt)** ซึ่งอาจมีปัญหาด้านประสิทธิภาพหรือความถูกต้อง โดยเฉพาะในระบบที่มีหลายหน่วยประมวลผลกลาง

แนวคิดของคำสั่ง Test-and-Set คือการดำเนินการที่**ไม่สามารถถูกขัดจังหวะได้ (atomic operation)** กล่าวคือ เป็นคำสั่งที่สามารถทดสอบค่าตัวแปรและปรับปรุงค่าได้ในคำสั่งเดียว ซึ่งป้องกันไม่ให้โพรเซสอื่นแทรกกลางเข้ามาได้ระหว่างกระบวนการ

การทำงานของคำสั่งนี้เริ่มจากโพรเซสที่ต้องการเข้าสู่ส่วนวิกฤติ จะทำการเรียกใช้ `TestAndSet()` เพื่อตรวจสอบว่ามีโพรเซสอื่นกำลังอยู่ในส่วนวิกฤติหรือไม่ หากพบว่าไม่มี โพรเซสจะดำเนินการเซตค่าล็อค เพื่อป้องกันไม่ให้โพรเซสอื่นเข้ามาในส่วนวิกฤติได้พร้อมกัน การดำเนินการนี้ทำให้สามารถรับประกันคุณสมบัติ **การไม่เกิดร่วม (mutual exclusion)** ได้แม้ในระบบที่มีหลายโพรเซส

```
boolean test_and_set (boolean *target)
   {
       boolean rv = *target;
       *target = TRUE;
       return rv:
   }
```

รูปที่ 3.9: รูปแบบของคำสั่ง Test-and-Set

ในหลายระบบคอมพิวเตอร์สมัยใหม่ ฮาร์ดแวร์มีคำสั่งพิเศษที่สามารถดำเนินการกับค่าของตัวแปรได้ในลักษณะ **อะตอมมิก (atomic)** ซึ่งหมายถึงไม่สามารถถูกขัดจังหวะได้ (non-interruptable) คำสั่งเหล่านี้ช่วยให้สามารถดำเนินการควบคุมการเข้าใช้ส่วนวิกฤติได้อย่างมีประสิทธิภาพ โดยไม่ต้องอาศัยกลไกหลายชั้นหรือเฉพาะเจาะจงกับสถาปัตยกรรมใดสถาปัตยกรรมหนึ่ง

ลักษณะสำคัญของ `TestAndSet()` คือการทำงานอย่างอะตอมมิก ซึ่งหมายความว่า แม้จะมีหลายโพรเซสจากหน่วยประมวลผลที่แตกต่างกันเรียกใช้งานพร้อมกัน คำสั่งนี้จะถูกดำเนินการเรียงลำดับทีละตัวในลักษณะที่ไม่ซ้ำซ้อน และไม่ทำให้เกิดการแทรกกลาง ส่งผลให้สามารถจัดการส่วนวิกฤติได้อย่างถูกต้องและปลอดภัย

โดยทั่วไป ค่าของตัวแปรล็อค (เช่น Boolean) จะถูกกำหนดให้เริ่มต้นเป็น `false` เพื่อบ่งชี้ว่าส่วนวิกฤติยังว่างอยู่ โพรเซสที่ต้องการเข้าสู่ส่วนวิกฤติจะทำการเรียกใช้คำสั่ง Test-and-Set เพื่อล็อคการเข้าถึง หากสำเร็จจึงสามารถเข้าสู่ส่วนวิกฤติได้ โครงสร้างของโพรเซส P_i ที่ใช้วิธีการนี้สามารถดูได้ในรูปที่ 1.10

```
    // Shared boolean variable lock, initialized to FALSE
    // Solution:

    do {
        while (test_and_set(&lock)) ; /* do nothing */
                /* critical section */
        lock = false;
                /* remainder section */
} while (true);
```

รูปที่ 3.10: โค้ดภาษาซีของโพรเซสในรูปแบบ Test-and-Set

คำสั่งสลับค่า

คำสั่งสลับค่า `Swap()` เป็นอีกหนึ่งคำสั่งระดับฮาร์ดแวร์ที่ใช้ในการควบคุมการเข้าถึง**ส่วนวิกฤติ (critical section)** ของโพรเซส ซึ่งมีวัตถุประสงค์เดียวกันกับคำสั่ง `TestAndSet()` คือการรับประกันว่าการเข้าถึงทรัพยากรที่ใช้ร่วมกันจะเกิดขึ้นเพียงโพรเซสเดียวในแต่ละช่วงเวลา โดยดำเนินการในลักษณะ **อะตอมมิก (atomic)** ซึ่งหมายถึงไม่สามารถถูกขัดจังหวะหรือแทรกกลางได้

แม้ว่า ทั้ง สอง คำ สั่ง จะ มี เป้า หมาย คล้าย กัน แต่ ลักษณะ การ ทำงาน ของ `Swap()` แตก ต่าง จาก `TestAndSet()` เล็ก น้อย กล่าว คือ `Swap()` จะ ทำการ สลับ ค่า ระหว่าง ตัวแปร สอง ค่า คือ **ตัวแปร global boolean** และ**ตัวแปร local boolean** ของโพรเซส โดยดำเนินการแบบอะตอมมิก ซึ่งช่วยให้สามารถใช้ควบคุมการล็อคได้อย่างปลอดภัยเช่นกัน

ตัวอย่างโค้ดของฟังก์ชัน `Swap()` แสดงไว้ในรูปที่ 1.11

```
Void Swap (boolean *a, boolean *b) {
        boolean temp = *a;
        *a = *b;
        *b = temp;
}
```

รูปที่ 3.11: ฟังก์ชัน `Swap()` ในโครงสร้างโค้ดภาษาซี

หากระบบคอมพิวเตอร์รองรับคำสั่ง `Swap()` แล้ว เราสามารถใช้คำสั่งนี้เพื่อควบคุมการเข้าถึงส่วนวิกฤติ โดยมีเงื่อนไขดังนี้:

- ตัวแปรล็อคแบบ **global boolean** จะถูกประกาศขึ้นและกำหนดค่าเริ่มต้นให้เป็น `false` เพื่อแสดงว่าส่วนวิกฤติยังไม่มีโพรเซสใช้งาน
- แต่ละโพรเซสจะมี **ตัวแปร local boolean** ของตนเอง (เช่น key)
- เมื่อโพรเซสต้องการเข้าสู่ส่วนวิกฤติ จะเรียกใช้คำสั่ง `Swap(lock, key)` ซึ่งจะทำการสลับค่าของ `lock` กับ `key` แบบอะตอมมิก
- หากหลังการสลับพบว่า `key` เป็น `false` แสดงว่าโพรเซสสามารถเข้าสู่ส่วนวิกฤติได้

โครงสร้างของโพรเซส P_i ที่ใช้ควบคุมการเข้าถึงส่วนวิกฤติด้วยคำสั่ง `Swap()` แสดงไว้ในรูปที่ 1.12

```
do {
        key = TRUE ;
            while (key = = TRUE )
                  Swap (&lock, &key) ,
                  // critical section
                Lock = FALSE;
                  // remainder section

} while  (TRUE) ;
```

รูปที่ 3.12: โค้ดภาษาซีแสดงการกีดกันโพรเซสโดยการเรียกใช้ฟังก์ชัน `Swap()`

3.6 โครงสร้างพื้นฐานสำหรับการซินโครไนซ์

ในการทำงานของโพรเซสที่ต้องประสานงานร่วมกันเพื่อดำเนินการบางอย่างในระบบคอมพิวเตอร์ การ**ประสานเวลา (synchronization)** อย่างเหมาะสมถือเป็นสิ่งสำคัญอย่างยิ่ง เพื่อให้การทำงานของโพรเซสเกิดความถูกต้อง ปลอดภัย และไม่เกิดข้อขัดแย้งกันในการใช้ทรัพยากรร่วมกัน

แม้ว่าก่อนหน้านี้จะมีการเสนอแนวทางเช่น การใช้ `Test-and-Set` หรือ `Swap()` ซึ่งเป็นคำสั่งระดับฮาร์ดแวร์ในการควบคุมการเข้าถึงส่วนวิกฤติ แต่แนวทางเหล่านั้นยังไม่สะดวกเพียงพอเมื่อต้องนำไปใช้กับระบบที่มีความซับซ้อนมากขึ้น ดังนั้นจึงมีการพัฒนาโครงสร้างที่มีประสิทธิภาพยิ่งขึ้นในการประสานเวลา ซึ่งหนึ่งในแนวคิดสำคัญคือ **เซมาฟอร์ (Semaphore)**

3.6.1 เซมาฟอร์ (Semaphore)

เซมาฟอร์ เป็นแนวคิดที่เสนอโดย**เอดส์เกอร์ ไดจสตร้า (Edsger W. Dijkstra)** ในปี ค.ศ. 1965 เพื่อใช้ในการควบคุมการเข้าถึงทรัพยากรและประสานเวลาการทำงานของโพรเซส โดยเซมาฟอร์เป็นตัวแปรชนิดจำนวนเต็ม (Integer) ซึ่งสามารถใช้ควบคุมจำนวนโพรเซสที่สามารถเข้าใช้งานทรัพยากรได้ในช่วงเวลาใดเวลาหนึ่ง

การทำงานของเซมาฟอร์ดำเนินการผ่านสองคำสั่งสำคัญ ได้แก่:

- **Wait(S)** หรือ `P(S)` — (Proberen): ใช้ตรวจสอบและลดค่าของเซมาฟอร์ หากค่า $S > 0$ จะลดค่า S ลง 1 และอนุญาตให้โพรเซสดำเนินการต่อ หาก $S = 0$ โพรเซสจะต้องรอ

- **Signal(S)** หรือ `V(S)` — (Verhogen): ใช้เพิ่มค่า S ขึ้น 1 เพื่อส่งสัญญาณว่าโพรเซสอื่นสามารถเข้าถึงทรัพยากรหรือดำเนินการได้

```
// S: wait() and signal()
// Originally called P() and V()

wait (S) {
        while S <= 0
                ; // no-op
          S--;
    }
signal (S) {
```

รูปที่ 3.13: โครงสร้างของคำสั่ง Signal และ Wait ใน Semaphore

คำสั่งทั้งสองนี้ต้องทำงานในลักษณะ**อะตอมมิก (Atomic)** กล่าวคือ เมื่อโพรเซสหนึ่งกำลังดำเนินการกับค่าเซมาฟอร์ จะต้องไม่มีโพรเซสอื่นสามารถแทรกกลางหรือเปลี่ยนแปลงค่าเดียวกันได้ เพื่อป้องกันเงื่อนไขการแย่งชิง (race condition)

เซมาฟอร์สามารถช่วยลดความซับซ้อนของการประสานเวลาระหว่างโพรเซส โดยทำหน้าที่เป็นตัวกลางในการควบคุมการเข้าถึงทรัพยากร เช่น หน่วยความจำ อุปกรณ์ I/O หรือพื้นที่วิกฤติอื่น ๆ ได้อย่างมีประสิทธิภาพ

แนวคิดสำคัญ:

- **แนวคิดของเซมาฟอร์:** สามารถอธิบายเปรียบเทียบได้โดยใช้ภาพของ**ขวดโหลลูกหิน** โดยจำนวนลูกหินที่อยู่ในขวดโหล ณ เวลาหนึ่ง แทนค่าของเซมาฟอร์ หากโพรเซสต้องการใช้ทรัพยากร ก็เปรียบได้กับการ “หยิบลูกหิน” ไปหนึ่งลูก (ทำ `P` หรือ `Wait`) หากมีลูกหินก็สามารถหยิบได้และดำเนินการต่อ แต่ถ้าไม่มีลูกหินเหลืออยู่ (ค่าเซมาฟอร์เป็น 0) โพรเซสจะต้องรอ

- **การทำงาน:** เมื่อโพรเซสหนึ่งเสร็จสิ้นการใช้งานทรัพยากร ก็จะทำการ “ใส่ลูกหินคืน” เข้าไปในขวดโหล (ทำ `V` หรือ `Signal`) เพื่อให้โพรเซสอื่นสามารถใช้ลูกหินต่อได้ หากมีหลายโพรเซสรออยู่ ระบบจะต้องมีวิธีเลือกโพรเซสที่ได้รับสิทธิ์ต่อไป เช่น การใช้นโยบายใครมาก่อนได้ก่อน (First-Come-First-Served) เพื่อความยุติธรรมในการเข้าถึง

3.6.2 การใช้งานเซมาฟอร์

เซมาฟอร์สามารถนำไปประยุกต์ใช้งานได้อย่างหลากหลาย โดยเฉพาะในการแก้ไขปัญหา**ส่วนวิกฤติ (critical section)** ของโพรเซสจำนวน n ตัว เพื่อให้แต่ละโพรเซสไม่สามารถเข้าสู่ส่วนวิกฤติพร้อมกันได้ ซึ่งจะช่วยรับประกันคุณสมบัติของ**การไม่เกิดร่วม (mutual exclusion)** ได้อย่างถูกต้อง

ในการใช้งานดังกล่าว โพรเซสทั้ง n ตัวจะร่วมกันใช้เซมาฟอร์ที่ชื่อ `mutex` (ย่อมาจาก *mutual exclusion*) ซึ่งถูกกำหนดค่าเริ่มต้นให้เป็น 1 เพื่อแสดงว่าส่วนวิกฤติยังว่างอยู่ โพรเซส P_i ใด ๆ ที่ต้องการ

เข้าสู่ส่วนวิกฤติจะต้องดำเนินการคำสั่ง `wait(mutex)` และเมื่อเสร็จสิ้นจะทำคำสั่ง `signal(mutex)` เพื่อให้โพรเซสอื่นสามารถเข้าสู่ส่วนวิกฤติได้ต่อไป โครงสร้างของโพรเซสแสดงไว้ในรูปที่ 1.14

```
do {
        wait(mutex);
                critical section
        signal(mutex);
                remainder section
        } while (true);
```

รูปที่ 3.14: การสร้าง Mutual Exclusion ด้วย Semaphore

นอกจากนี้ เซมาฟอร์ยังสามารถใช้ประสานการทำงานของโพรเซสที่มีความสัมพันธ์ระหว่างกัน เพื่อควบคุม**ลำดับของการดำเนินการ (process synchronization)** ได้อีกด้วย ตัวอย่างเช่น สมมติให้มีโพรเซสสองตัวคือ P1 และ P2 โดยโพรเซส P1 มีขั้นตอนการทำงานที่สถานะ S1 และโพรเซส P2 มีขั้นตอนการทำงานที่สถานะ S2 หากต้องการให้ S2 ทำงานได้เฉพาะเมื่อ S1 ทำงานเสร็จแล้วเท่านั้น เราสามารถใช้เซมาฟอร์ร่วมกันชื่อว่า `synch` โดยมีแนวทางดังนี้:

- กำหนดค่าเริ่มต้นของ `synch` ให้เป็น 0
- ภายในโพรเซส P1 ให้เพิ่มคำสั่ง `signal(synch)` หลังจากเสร็จสิ้นสถานะ S1
- ภายในโพรเซส P2 ให้เพิ่มคำสั่ง `wait(synch)` ก่อนเข้าสู่สถานะ S2

ด้วยวิธีการนี้ โพรเซส P2 จะไม่สามารถดำเนินการที่ S2 ได้จนกว่า P1 จะดำเนินการที่ S1 เสร็จและส่งสัญญาณด้วย `signal(synch)` ทำให้สามารถควบคุมลำดับของการทำงานระหว่างโพรเซสได้อย่างมีประสิทธิภาพ

3.6.3 การสร้างเซมาฟอร์

ในการใช้งานเซมาฟอร์แบบดั้งเดิม การแก้ไขปัญหาการกีดกัน (mutual exclusion) อาจนำไปสู่ปัญหาใหม่ที่เรียกว่า **การวนเวียนรอคอย (busy waiting)** กล่าวคือ เมื่อโพรเซสหนึ่งกำลังทำงานอยู่ในส่วนวิกฤติและมีโพรเซสอื่นต้องการเข้าใช้งานด้วย โพรเซสนั้นจะต้องวนลูปรอโดยตลอด ซึ่งทำให้เสียเวลาของหน่วยประมวลผลกลางโดยเปล่าประโยชน์ เพราะไม่สามารถนำทรัพยากรนั้นไปใช้ประโยชน์กับโพรเซสอื่นได้ ซึ่งกรณีนี้เราจะเรียกว่า **Spinlock** โดยเฉพาะในระบบแบบมัลติโพรเซสเซอร์ Spinlock อาจมีข้อดีบ้างในกรณีที่ระยะเวลาการล็อคสั้น เพราะไม่ต้องทำ *context switch* แต่หากระยะเวลาการล็อกยาว การวนลูปรอจะทำให้ประสิทธิภาพของระบบลดลงอย่างมาก

แนวทางการปรับปรุงเพื่อหลีกเลี่ยง busy waiting

เพื่อแก้ไขปัญหาดังกล่าว เราสามารถปรับปรุงนิยามของคำสั่ง `wait()` และ `signal()` ใหม่ โดยมีแนวทางดังนี้:

- หากโพรเซสใดทำคำสั่ง `wait()` แล้วพบว่าเซมาฟอร์มีค่าน้อยกว่าหรือเท่ากับ 0 โพรเซสนั้นจะถูก **ระงับการทำงาน (block)** โดยไม่วนลูปรอ แต่จะถูกส่งไปอยู่ใน **แถวคอย (waiting queue)** และเปลี่ยนสถานะเป็น `waiting state`

- เมื่อมีโพรเซสอื่นทำคำสั่ง `signal()` กับเซมาฟอร์ตัวเดียวกัน โพรเซสที่อยู่ในแถวคอยจะถูก**ปลุก (wake up)** และเปลี่ยนสถานะเป็น `ready state` เพื่อรอการจัดสรรให้ทำงานต่อ

การดำเนินการเช่นนี้ทำให้หน่วยประมวลผลกลางสามารถสลับไปทำงานกับโพรเซสอื่นได้ทันที แทนที่จะเสียเวลาไปกับการวนลูปที่ไม่เกิดประโยชน์

โครงสร้างของเซมาฟอร์แบบใหม่

เซมาฟอร์แบบใหม่นี้สามารถนิยามได้เป็นโครงสร้างในภาษาซี ดังนี้:

Listing 3.1: Semaphore

```
typedef struct {
    int value;
    struct process *L;
} semaphore;
```

ตัวแปรเซมาฟอร์ประกอบด้วย:

- `value` — ตัวแปรจำนวนเต็มที่แสดงสถานะการใช้ทรัพยากร
- `L` — แถวคอยของโพรเซสที่รอการใช้ทรัพยากร (waiting queue)

เมื่อโพรเซสหนึ่งทำคำสั่ง `wait(S)` แล้วไม่สามารถเข้าถึงทรัพยากรได้ โพรเซสนั้นจะถูกเพิ่มเข้าไปในแถวคอย `L` และหยุดการทำงานชั่วคราว (block) ส่วนเมื่อโพรเซสอื่นทำคำสั่ง `signal(S)` โพรเซสที่อยู่ต้นแถวจะถูกปลุกให้กลับเข้าสู่สถานะพร้อมและกลับมาทำงานต่อได้

```
void wait(semaphore S) {
        S.value - -;
            if (S.value < 0) {
                add this process to S.L (list);
                block();
              }
    }
void signal(semaphore S) {
        S.value ++;
            if (S.value <= 0) {
              remove a process P from S.L (list);
              wakeup(P);
            }
    }
```

รูปที่ 3.15: ตัวอย่าง Semaphore เป็นโครงสร้างโค้ดในภาษาซี

`block(P)` คือคำสั่งที่ใช้ระงับการทำงานของโพรเซส P ส่วน `wakeup(P)` คือคำสั่งที่ใช้ปลุกโพรเซสให้กลับมาทำงานใหม่ โดยลักษณะนี้ทำให้**ค่าของเซมาฟอร์สามารถเป็นค่าติดลบได้** ซึ่งหมายถึงจำนวนโพรเซสที่รอการใช้เซมาฟอร์ตัวนั้น

การป้องกันปัญหาการเข้าถึงพร้อมกัน

การเข้าถึงเซมาฟอร์โดยหลายโพรเซสพร้อมกันอาจทำให้เกิด**ปัญหาเขตวิกฤติ (critical section problem)** ดังนั้น ต้องมีแนวทางในการป้องกัน เช่น:

1. **ในระบบที่มีหน่วยประมวลผลเดียว (single processor):** ป้องกันโดยการห้ามขัดจังหวะ (disable interrupt) ขณะที่กำลังดำเนินการคำสั่ง `wait()` และ `signal()` เพื่อให้คำสั่งทำงานเสร็จสิ้นโดยไม่ถูกรบกวน

2. **ในระบบที่มีหลายหน่วยประมวลผล (multiprocessor):** การห้ามขัดจังหวะไม่เพียงพอ เพราะโพรเซสอาจรันอยู่ในหน่วยประมวลผลอื่นได้ จึงต้องใช้เทคนิคการแก้ไขส่วนวิกฤติ (critical section solution) เช่น การใช้ `Test-and-Set` หรือ `Compare-and-Swap` เพื่อป้องกันไม่ให้โพรเซสสองตัวเข้าถึงเซมาฟอร์ตัวเดียวกันพร้อมกันได้

3.6.4 เซมาฟอร์แบบทวิภาค (Binary Semaphore)

จากที่กล่าวมาในหัวข้อก่อนหน้า เซมาฟอร์ที่ใช้งานทั่วไปมักเป็น**เซมาฟอร์แบบนับ (counting semaphore)** ซึ่งสามารถมีค่าเป็นจำนวนเต็มใด ๆ ก็ได้ ใช้ในการควบคุมโพรเซสหลายตัวที่ต้องการใช้ทรัพยากรร่วมกันจำนวนมากกว่า 1 หน่วย

แต่ในกรณีที่ต้องการควบคุม**การเข้าถึงแบบครั้งละหนึ่งโพรเซส** เท่านั้น สามารถใช้**เซมาฟอร์แบบทวิภาค (binary semaphore)** ซึ่งเป็นเซมาฟอร์ชนิดพิเศษที่มีค่าตัวแปรได้เพียงสองค่า คือ 0 และ 1 เท่านั้น โดย:

- ค่า `1` หมายถึง สถานะ ”ว่าง” (Available)
- ค่า `0` หมายถึง สถานะ ”ถูกใช้งาน” (Unavailable)

เซมาฟอร์แบบทวิภาคมีโครงสร้างที่เรียบง่าย และเหมาะกับการนำมาใช้สร้างกลไกการกีดกัน (mutual exclusion) สำหรับโพรเซสสองตัว หรือการจัดลำดับการเข้าถึงส่วนวิกฤติ โดยเฉพาะในสถานการณ์ที่ไม่จำเป็นต้องนับจำนวนการเข้าถึงซ้ำ

โครงสร้างของการสร้างเซมาฟอร์แบบทวิภาคสามารถดูได้จากรูปที่ 1.16 โดยกำหนดให้ตัวแปรเซมาฟอร์ S1 และ S3 เริ่มต้นเป็น 1 ส่วน S2 เริ่มต้นเป็น 0

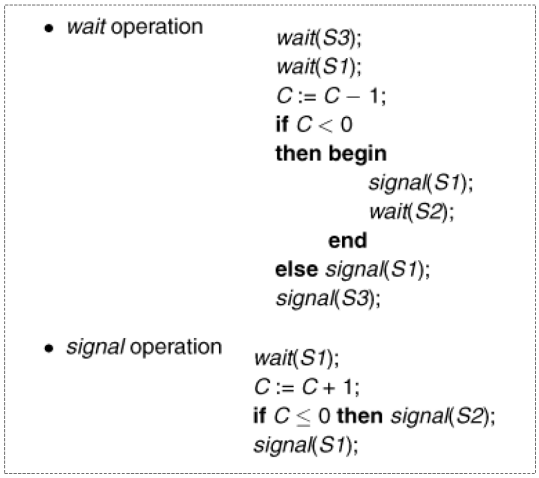
```
Var     S1: binary-semaphore;
        S2: binary-semaphore;
        S3: binary-semaphore;
        C: integer;
```

รูปที่ 3.16: โครงสร้างของการใช้ Semaphore แบบทวิภาค

ส่วนการใช้งานจริงในลักษณะการเข้าถึงส่วนวิกฤติของโพรเซสสองตัว สามารถดูได้จากรูปที่ 1.17 ซึ่งแสดงให้เห็นลำดับการดำเนินการผ่านคำสั่ง `wait()` และ `signal()` โดยอิงกับเซมาฟอร์แบบทวิภาค

หลักการทำงานมีรายละเอียดดังนี้:

- ก่อนที่โพรเซสจะดำเนินการลดค่าของตัวแปรเซมาฟอร์ C จะต้องเรียก `wait(S1)` เพื่อเข้าสู่เขตวิกฤติในการตรวจสอบและปรับค่าของ C



รูปที่ 3.17: รูปแบบของการใช้งาน Semaphore แบบทวิภาค

- หากค่าที่ลดของ C น้อยกว่า 0 จะต้องดำเนินการ `wait(S2)` ต่อทันที โดยมีการปลดล็อก `S1` เพื่อให้โพรเซสอื่นสามารถเข้าได้
- หากค่าของ C มากกว่าหรือเท่ากับ 0 หลังจากลดค่าแล้ว ก็สามารถปลดล็อก `S1` ได้โดยไม่ต้องเรียก `wait(S2)`
- ในคำสั่ง `signal()` เมื่อมีการเพิ่มค่าของ C แล้ว หากค่าที่ได้ใหม่ยังคงน้อยกว่าหรือเท่ากับ 0 แสดงว่ายังมีโพรเซสรออยู่ในแถวคอย จึงต้อง `signal(S2)` เพื่อปลุกโพรเซสที่รออยู่ให้กลับมาทำงาน แล้วจึงปลดล็อก `S1`

โดยทั่วไป สถานะ S1 และ S2 ในโครงสร้างนี้อาจใช้แทนโพรเซส P1 และ P2 ตามลำดับ ทั้งหมดนี้เป็นการประสานการเข้าถึงส่วนวิกฤติอย่างมีประสิทธิภาพสำหรับโพรเซสสองตัว ด้วยการใช้**เซมาฟอร์แบบทวิภาค**